Exp 1:

QUESTIONS 5 / 5 completed

Merge Sort Playlist

MERGE SORT PLAYLIST

A music app wants to merge-sort a playlist of N song durations (in seconds) into ascending order for a smooth listening experience. Write a program implementing merge sort.
(1) Read N and the durations.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

void printArray(int arr[], int l, int r) {
    for (int i = l; i <= r; i++)
        printf("%d ", arr[i]);
}

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    printf("Merging: ");
    printArray(arr, l, m);
    printf("-> ");
```

INPUT

```
6
180 210 95 260 150
300
```

Exp 2:

QUESTIONS 5 / 5 completed

Quick Sort Pivot Tr

QUICK SORT PIVOT TRACE

A tournament organizer wants to rank N players by score using quicksort and wants to see which score is chosen as the pivot at each partitioning step. Write a program implementing quicksort (last element as pivot).

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high, int n) {
    int pivot = arr[high]; // Last element as pivot
    int i = (low - 1);

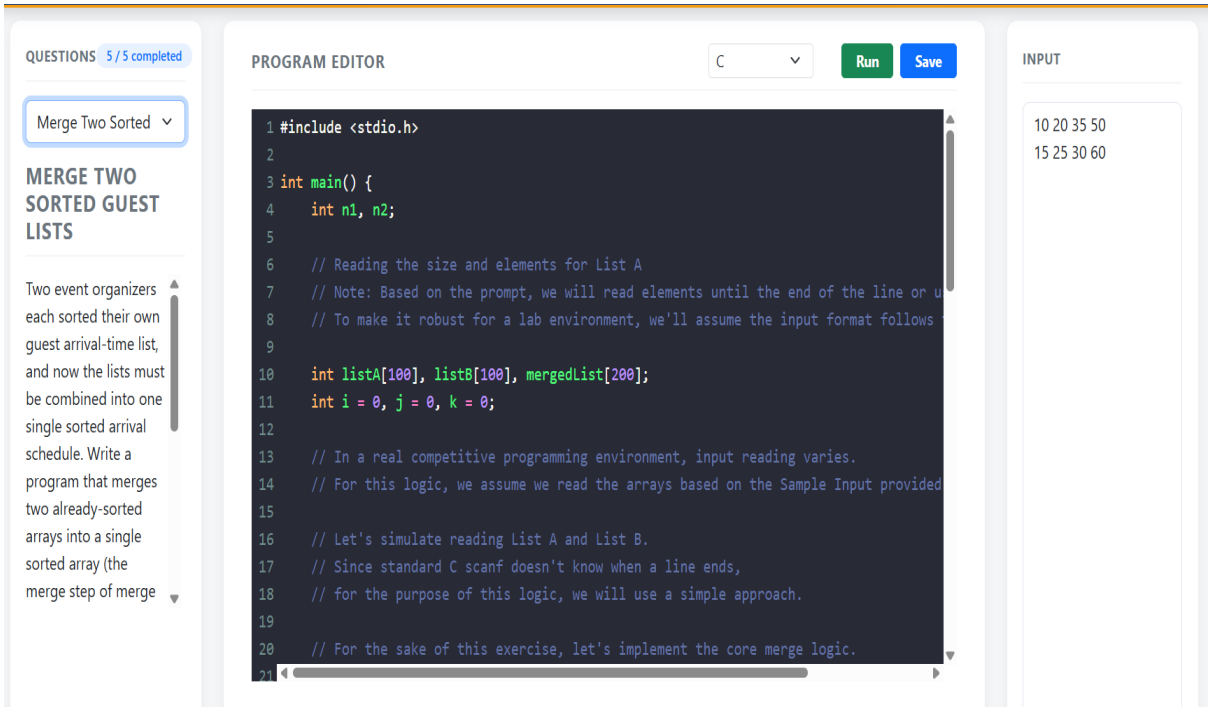
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);

    // Print the pivot and the array after partitioning
```

INPUT

```
6
10 80 30 90 40 50
```

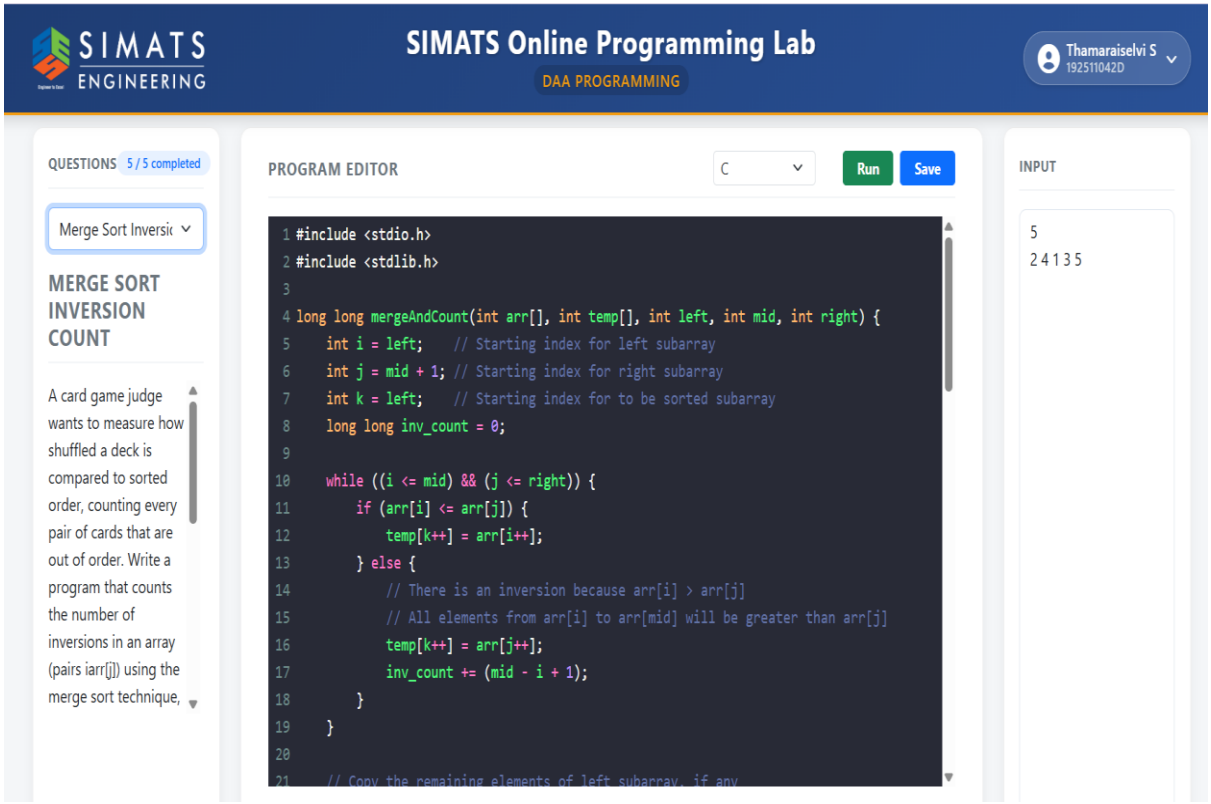
Exp 3:


QUESTIONS 5 / 5 completed
Merge Two Sorted
MERGE TWO SORTED GUEST LISTS
Two event organizers each sorted their own guest arrival-time list, and now the lists must be combined into one single sorted arrival schedule. Write a program that merges two already-sorted arrays into a single sorted array (the merge step of merge
PROGRAM EDITOR
C
Run
Save
#include <stdio.h>
int main() {
int n1, n2;
// Reading the size and elements for List A
// Note: Based on the prompt, we will read elements until the end of the line or u
// To make it robust for a lab environment, we'll assume the input format follows
int listA[100], listB[100], mergedList[200];
int i = 0, j = 0, k = 0;
// In a real competitive programming environment, input reading varies.
// For this logic, we assume we read the arrays based on the Sample Input provided
// Let's simulate reading List A and List B.
// Since standard C scanf doesn't know when a line ends,
// for the purpose of this logic, we will use a simple approach.
// For the sake of this exercise, let's implement the core merge logic.
INPUT
10 20 35 50
15 25 30 60

Exp 4:


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Thamaraiselvi S
192511042D
QUESTIONS 5 / 5 completed
Quick Sort Exam Ra
QUICK SORT EXAM RANKING
A college wants to rank N students by exam score from highest to lowest using quicksort for the merit list. Write a program implementing quicksort to sort an array in descending order.
(1) Read N and the
PROGRAM EDITOR
C
Run
Save
#include <stdio.h>
// Function to swap two elements
void swap(int* a, int* b) {
int t = *a;
*a = *b;
*b = t;
}
// Partition function for descending order
int partition(int arr[], int low, int high) {
int pivot = arr[high]; // Choosing the last element as pivot
int i = (low - 1);
for (int j = low; j <= high - 1; j++) {
// For descending order, we check if current element is GREATER than pivot
if (arr[j] > pivot) {
i++;
swap(&arr[i], &arr[j]);
}
}
INPUT
5
72 88 65 91 79

Exp 5:


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Thamaraiselvi S
192511042D
QUESTIONS 5 / 5 completed
Merge Sort Inversi
MERGE SORT INVERSION COUNT
A card game judge wants to measure how shuffled a deck is compared to sorted order, counting every pair of cards that are out of order. Write a program that counts the number of inversions in an array (pairs iarr[jj) using the merge sort technique,
PROGRAM EDITOR
C
Run
Save
#include <stdio.h>
#include <stdlib.h>
long long mergeAndCount(int arr[], int temp[], int left, int mid, int right) {
int i = left; // Starting index for left subarray
int j = mid + 1; // Starting index for right subarray
int k = left; // Starting index for to be sorted subarray
long long inv_count = 0;
while ((i <= mid) && (j <= right)) {
if (arr[i] <= arr[j]) {
temp[k++] = arr[i++];
} else {
// There is an inversion because arr[i] > arr[j]
// All elements from arr[i] to arr[mid] will be greater than arr[j]
temp[k++] = arr[j++];
inv_count += (mid - i + 1);
}
}
// Copy the remaining elements of left subarray, if any
INPUT
5
2 4 1 3 5